

软件工程导论

CH0:计算机技术史

软件工程提出时间：1968年 ☆

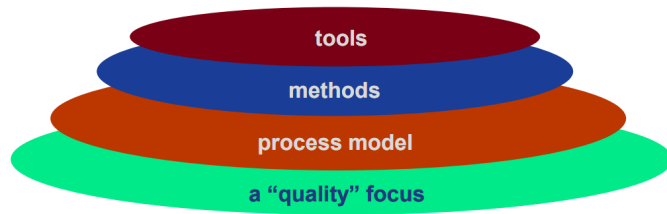
CH1:软件的本质

- 软件包含 ☆
 - **程序**：执行时提供所需特性、功能和性能的指令
 - **数据结构**：使程序能够充分操纵信息的数据结构
 - **文档**：描述程序的操作和使用的文档
- 软件相较于硬件的特性 ☆
 - 开发
 - 加人可能无效——交流困难
 - 经常修改
 - 成本在设计而非生产
 - **退化**
 - 修改后产生新的错误
 - 不再适应需求
 - 定制
 - 虽然朝着基于构件的模式发展，但还是定制的
 - 复杂
- 遗留软件
 - 特点
 - 生命周期长
 - 业务关键
 - 质量差
 - 修改原因
 - 新的计算环境或技术需求
 - 新的商业需求
 - 与其他系统和数据库的交互
 - 新的网络环境

CH2:软件工程的概念

- 软件危机 ☆：落后的 软件 生产方式无法满足迅速增长的 计算机软件 需求。
- 软件工程
 - IEEE标准 ☆：（1）在软件的开发、操作和维护中应用系统的、规范的、可量化的方法（2）在（1）中方法的研究

- 层次化的工具



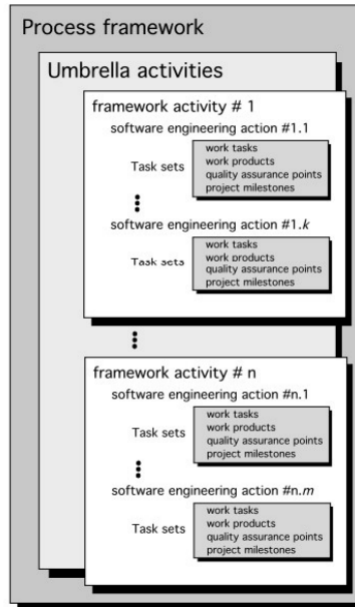
Software Engineering

- 软件生命周期：软件有一个孕育、诞生、成长、成熟、衰亡的生存过程。这个过程即为计算机软件的生命周期
 - 意义：
 - 对问题进行分解
 - 提供一个公共的框架，以便软件人的相互交流
- 软件工程实践
 - 理解问题
 - 利益相关者？
 - 未知的数据、功能、特性、行为？
 - 能否划分问题？
 - 建立模型？
 - 计划解决方案
 - 相似问题？
 - 子问题？
 - 快速设计解决方案？
 - 实施计划
 - 计划 and 设计一致？
 - 每个组成部分正确？
 - 检查结果
 - 每个部分都进行了测试？
 - 产生了与需求一致的结果？

CH3 软件过程结构

- 软件过程模型
 - 过程：在工作产品构建过程中需要完成的活动
 - 框架或模型：活动的顺序以及相互关系
 - 目标：标准化、可预测性、生产力、高产品质量、计划时间和预算要求的能力
- 软件过程通用框架 ☆
 -

Software process



○ 框架活动 ☆ ☆

■ 沟通

- 应用需求确认
- 软件规约

■ 策划

- 计划
- 估算
- 安排

■ 建模

- 架构设计
- 细节设计

■ 构建

- 实现
- 集成
- 测试

■ 部署

○ 庇护活动（普适性活动）：补充实现框架活动

- 软件项目跟踪和控制
- 风险管理
- 软件质量保证
- 技术评审
- 测量
- 软件配置管理
- 可复用管理
- 工作产品的准备和生产

○ 明确任务集：定义了完成软件工程行动目标的实际工作

● 过程模式：描述软件过程中的问题并提供解决方案

- 步骤模式：定义与过程的框架活动相关的问题
- 任务模式：定义与软件工程操作或工作任务相关的问题
- 阶段模式：定义随过程发生的框架活动的顺序，即使活动的整体流程本质上是迭代的。

● CMMI软件过程模型示例 ☆

1. **初始**：过程不可预测，缺乏管理

2. **已管理**：建立了基本的项目管理过程
3. **已定义**：软件过程标准化
4. **量化管理**：过程定量并控制
5. **持续优化**：关注过程改进

CH4 软件过程模型

- 规范的软件过程模型 ☆

- 框架活动
- 软件工程活动
- 任务
- 产品
- 质量保证
- 变更控制

- 常见过程模型 ☆

1. 瀑布模型 ☆ ☆

- **沟通->策划->建模->构建->部署**

- 特点：

- 从上一项活动中接受该项活动的工作成果(工作产品)，作为输入
- 利用这一输入实施该项活动应完成的内容
- 给出该项活动的工作成果，作为输出传给下一项活动
- 对该项活动实施的工作进行评审。若其工作得到确认，则继续下一项活动

- 优点：

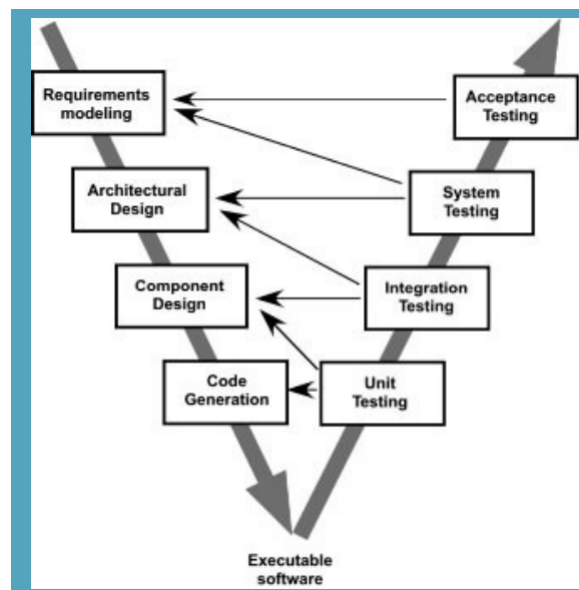
- 强调开发的阶段性
- 强调早期计划及需求调查
- 强调产品测试

- 缺点：

- 瀑布模型过于依赖早期进行的唯一一次需求调查，不能适应需求的变化
- 瀑布模型是单一流程，开发中的经验教训不能反馈应用于本产品的过程

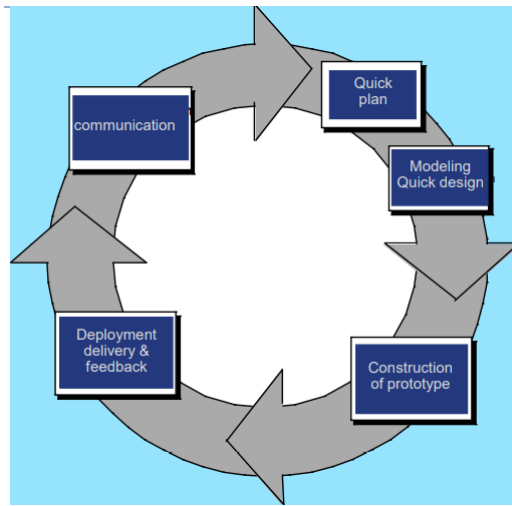
2. V模型 ☆ ☆

- 示意图



3. 演化模型（原型）

- 示意图



- 适用范围

- 需求经常改变
- 时间紧迫
- 只能确定核心需求

- 特征

- 它是一个可实际运行的系统
- 它没有固定的生存期。它可能被扔掉，或者作为最终产品的一部分
- 从需求分析到最终产品都可作原型，即可为不同目标作原型
- 它必须快速、廉价
- 它是迭代过程的集成部分，即每次经用户评价后修改、运行，不断重复双方认可

- 优点

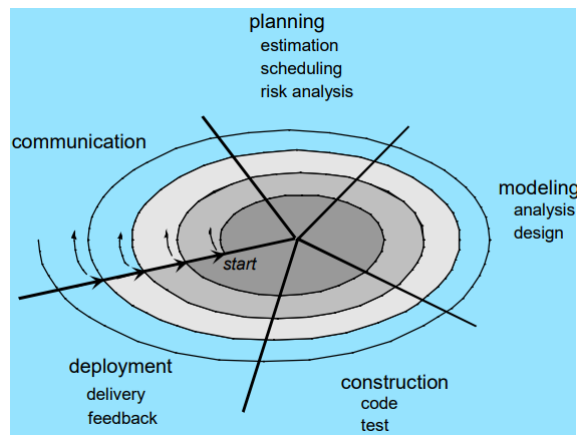
- 一开始就能弄清楚所有的产品需求，或者至少可以帮助引导出高质量的产品需求
- 可在项目早期就获取项目的相关数据，尽早进行风险管理和配置管理
- 心理上，开发人员早日见到产品的雏形，是一种鼓舞
- 使用户可以在新的一批功能开发测试后，立即参加验证，以便提供非常有价值的反馈
- 可使销售工作有可能提前进行，因为可以在产品开发的中后期取得包含了主要功能的产品原型去向各户作展示和试用

- 缺点

- 如果缺乏严格的过程管理的话，这个生命周期模型很可能退化成为一种原始的无计划的“试——错——改”的循环模式
- 心理上，可能产生一种影响尽最大努力的想法，认为虽然不能完成全部功能，但还是造出了一个有部分功能的产品
- 如果不加控制地让用户接触开发中尚未测试稳定的功能，可能对开发人员及用户都产生负面的影响

4. 演化模型（螺旋）

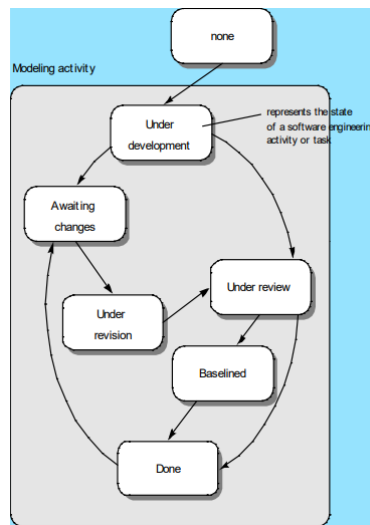
- 示意图



- 瀑布模型和演化模型(原型)的结合，并增加了风险分析
- 优点
 - 强调严格的全过程风险管理
 - 强调各开发阶段的质量
 - 提供机会检讨项目是否有价值继续下去
- 缺点
 - 必须引入非常严格的风险识别，风险分析和风险控制，这对风险管理的技能水平提出了很高的要求。
 - 需要人员，资金和时间的大量投入

5. 协调模型

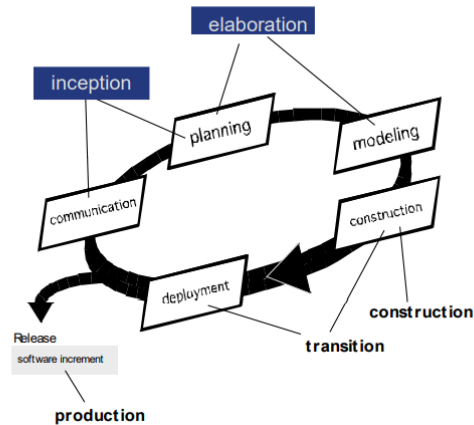
- 示意图



6. 增量模型

- 适应范围
 - 需求经常变化的软件开发
 - 市场急需而开发人员和资金不能在设定的市场期限之前实现一个完善的产品的软件开发
- 内容
 - 从给定需求开始，通过构造一系列中间版本来实施开发活动，依次类推，直到系统完成
 - 每一个中间版本都是需求分析、设计、编码和测试的过程
 - 某些中间版本的开发可以并行进行。
- 特征
 - 融合了线性顺序模型的基本成分和原型的迭代特征
 - 是随着日程时间的进展而交错的线性序列
 - 与原型不一样的地方是强调每个增量均发布一个可操作产品

- 最典型的应用是同一个产品的不同项目(合同、用户)版本
- 统一过程 (UP) ☆ ☆
 - 特征:
 - 用例驱动
 - 以架构为核心
 - 迭代并且增量
 - 软件过程与UML紧密结合
 - 时间轴被分成4个阶段: **起始、细化、构建、交付**



- 处理 workflow
 - 业务建模
 - 需求
 - 分析和设计
 - 实现
 - 测试
 - 部署
- 支持 workflow
 - 配置管理
 - 项目管理
 - 环境
- 优点
 - 提高了团队生产力
 - 针对所有关键的开发活动为每个开发成员提供了必要的准则、模板和工具指导
 - 它建立了简洁和清晰的过程结构，为开发过程提供较大的通用性
- 缺点
 - RUP在理论上，是比较理想的，但在实际应用上，还需要更多的工具的支持和普及推广工作。

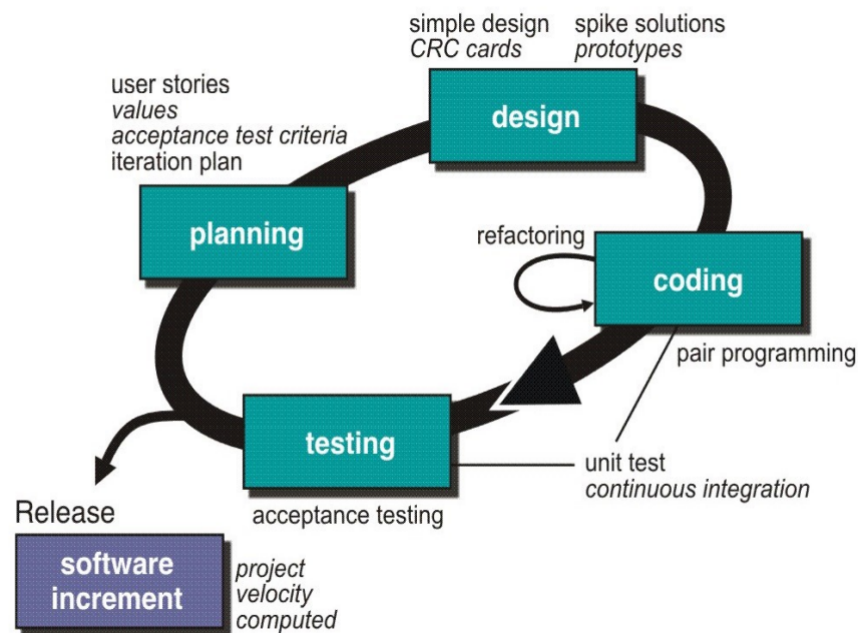
CH5 敏捷开发

- 敏捷宣言: ☆ ☆
 - 个体和互动 高于 流程和工具
 - 工作的软件 高于 详尽的文档
 - 客户合作 高于 合同谈判
 - 响应变化 高于 遵循计划
- 什么是敏捷开发 ☆

- 对变化的有效（快速和适应性）响应
- 所有利益相关者之间的有效沟通
- 将客户加入到团队中
- 团队组织
- 快速、增量地交付软件
- 敏捷过程 ☆
 - 由客户对所需内容的描述（场景）驱动
 - 计划周期短
 - 迭代开发软件、注重构建
 - 多个软件增量
 - 随着变化的发生而适应
- 敏捷与传统瀑布型模式的最大区别——价值驱动 ☆ ☆

敏捷基于这种方式，可以实现研发过程的持续高 可视性、高可适应性，更早且持续产出业务价值，更早发现和解决风险

- 极限编程（Extreme Programming）
 - 示意图



- XP planning
 - 从创建用户故事开始
 - 敏捷团队评估每个故事并分配成本
 - 将故事分组以获得可交付的增量
 - 承诺在交付日期做出
 - 在第一次增量项目速度(速度)是用来帮助定义为其他增加后续的交付日期
- XP designing
 - 遵循KIS原则(保持简单)
 - 鼓励使用CRC卡(标准索引卡集合)
 - 对于困难的设计问题，建议创建spike(探针)解决方案——一个设计原型
 - 鼓励重构——对内部程序设计的迭代改进
- XP coding
 - 测试驱动编程
 - 结对编程
- XP testing

- 强调daily building, 冒烟测试
 - 强调验收测试, 回归测试
- Scrum
 - 特点 ☆
 - 开发工作被划分为“包”
 - 随着产品的构建, 测试和文档记录正在进行中
 - 工作发生在“冲刺”中, 源自现有需求的“积压”
 - 会议时间很短, 有时在没有椅子的情况下进行
 - “demo”在分配的时间盒内交付给客户
 - 框架
 - 团队模型
 1. Scrum Master
 2. Product Owner
 3. 团队
 - 工件
 1. 产品待办项
 2. 冲刺待办项
 3. 产品增量
- 敏捷开发管理方法——看板 ☆

看板信号由下游向上游传递, 拉动上游的生产活动。拉动的源头是最下游的客户价值, 也就是客户订单或需求。

CH6 开发过程中人的影响

CH7 理解需求

1. 系统工程

1. 基于计算机的系统

- 通过处理信息 来完成某些预定义目标而组织在一起的元素 的组合
- 对于用户而言有意义的是可以达到预期目标 的系统(完整的软硬件解决方案)而不是单一的系统
- 组成基于计算机系统的元素主要有: 软件、硬件、人员、数据库(及其它系统软件)、文档和规程(Procedure)

2. 系统的层次结构

- 任何系统都处在一个更大的系统之中, 形成系统 的层次结构

2. 需求工程

1. 过程活动 ☆ ☆

1. 起始——与客户建立初步交流

- 确定利益相关者
- 问题和解决的大致方向

2. 导出——通过各种途径获取用户的需求

- 识别问题
- 提出解决要素
- 协商方法
- 确认放案

3. 精化——创建数据、功能和行为要求的分析模型
 - 建立分析模型中的元素
 - 基于场景的元素——用例图
 - 基于类的元素——类图
 - 基于行为的元素——状态图
 - 基于数据流的元素——数据流图
4. 协商——开发者与客户之间就所要交付的系统的功能、性能、交付时间等问题进行估算和协商
 - 识别关键的利益相关者
 - 确认利益相关者的满足条件
 - 协商
5. 规格说明——根据需求调查和需求分析的结果，进一步定义准确无误的产品需求，产生《产品需求规格说明书》
6. 确认——进一步确认规格说明
 - 一致性
 - 完整性
 - 非歧义性
7. 管理
 - 查找bug
 - verification——软件是否按团队要求运行
 - validation——软件是否按甲方要求运行
 - 系统是否满足业务目标
 - 向利益相关者提供进度
3. 质量功能部署（QFD）☆
 1. 功能部署——确定系统功能的价值
 2. 信息部署——确定数据对象和事件
 3. 任务部署——确定系统的行为
 4. 价值分析——确定需求的优先级
4. 非功能需求（NFR）
 1. 质量属性
 2. 性能属性
 3. 安全属性
 4. 其他约束
5. 用例（Use-Cases）——描述用户使用系统的场景

CH8~10 需求建模

1. 需求分析
 1. 总体目标
 - 客户需要什么
 - 为软件设计奠定基础
 - 定义在软件完成后可以被确认的一组需求
 2. 经验和原则
 - 关注的层次较高，不要陷入细节
 - 模型中的每个元素都应该对理解系统有帮助
 - 基础结构和非功能模型推迟到设计阶段再考虑
 - 最小化系统的耦合

- 确认需求模型为所有利益相关者带来价值
- 使模型尽量简单

3. 领域分析

- 确定要调查的相关领域
- 收集代表性的成果
- 分析收集到的内容
- 为项目开发分析模型

2. 需求建模

1. 分析方法——建模☆☆☆

○ 基于场景建模——用例图☆

1. 创建初始用例——写什么

- 起始、导出提供开始的灵感
- 需求收集会议、QFD等确定利益相关者、问题范围、整体运行目标、优先级、功能需求、系统处理的对象
- 开发用例时列出参与者执行的功能

2. 精化初始用例——写多少（可以使用活动图）

- 某一状态时有多种动作
- 异常情况
- 验证等其他情况

3. 完善用例

- 用例先用叙述的形式编写，然后再使用规格化的语言和图
- 查看场景是否有重复、缺失

○ 基于数据建模

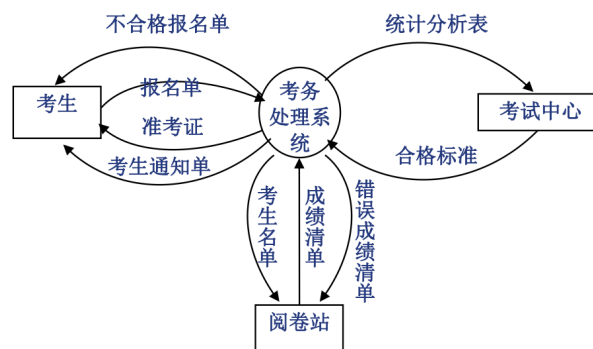
○ 基于类建模☆☆

1. 结构化分析——将数据和过程转换成独立的实体（具有分层结构）☆

- 数据对象用属性+关系来描述
- 过程用在系统中流转数据的方式来描述
- 示例图

Structured Analysis

❖Example: 考务处理系统顶层图



2. 面向对象分析（基于类的建模）

- 类的定义+协作方式（识别分析类）

1. 元素

- 出现在系统中的对象
- 对象上的操作
- 类之间的关系

- 类和类的协作

2. 识别分析类

- 提取名词

- 潜在类的判别：☆

1. 保留信息 —— 记录该类的信息才能保证系统正常工作
2. 需要操作 —— 对该类具有一组操作用以改变类的属性值
3. 多个属性 —— 只有一个属性的类最好作为另一个类的某个属性
4. 公共属性 —— 该类具有一组属性适用于所有实例
5. 公共操作 —— 该类具有一组操作适用于所有实例
6. 必须要求 —— 问题空间外的实体或者解决问题中产生的信息

- 潜在类的分类

1. 外部实体 —— 产生或使用系统信息
2. 事物 —— 信息的一部分
3. 事件 —— 系统中发生的操作
4. 角色 —— 和系统交互的人员
5. 组织单元 —— 与某个应用有关
6. 场地 —— 问题的环境和功能
7. 结构 —— 对象的类或相关的类

- 画出类图 —— 简单->属性、操作->约束

——>操作包含：增删改、计算、查、监视

- CRC卡（类+责任+协作者）

1. 类的分类

- 实体类 —— 明确的事物
- 边界类 —— 与用户交互的接口
- 控制类 —— 管理系统的数据走向

2. 职责（属性+操作）

- 智能系统应该分布在所有类中
- 职责说明应该具有普遍性（可以继承）
- 信息和操作应该放在同一个类中
- 有关同一事物的应当作为同一个类
- 适当时，职责可以被共享

3. 协作

- 类有两种方式实现其职责 —— 1、使用自身的操作控制自身的属性 2、和其他类协作

- 通用的协作关系

- is-part-of
- has-knowledge-of
- depends-upon

- 评审CRC模型

1. 评审员拿到不存在协作关系的CRC卡
2. 分类管理用例场景
3. 评审组长阅读用例，看到已命名的对象后发token给对应的人
4. 令牌传递时被传递者描述卡上的职责，讨论确定是否满足用例
5. 如果不能满足用例则修改卡片

- 分析包 —— 把相同功能的类放到一起

- 基于数据流建模

- 基于行为建模（列出系统的状态、表面系统的状态如何转变、画状态图或顺序图）
 - 创建行为模型的步骤
 1. 评估所有用例以充分了解系统内的交互顺序
 2. 识别驱动交互序列的事件，并了解这些事件与特定对象的关系
 3. 为每个用例创建一个序列
 4. 为系统构建状态图
 5. 审查行为模型以验证准确性和一致性
 - 状态表现
 - 在行为建模的上下文中，必须考虑两种不同的状态表征：
 1. 系统执行其功能时每个类的状态
 2. 系统执行其功能时从外部观察到的系统状态
 - 类状态具有被动和主动两种特征
 - 被动——对象所有属性的当前状态
 - 主动——对象在进行持续的转换或处理时的当前状态
 - 系统的状态
 - 相关词的含义
 - state——一组可观察到的情况，表征系统在给定时间的行为
 - state transition——从一个状态到另一个状态的移动
 - event——导致系统表现出某种可预测的行为形式的事件
 - action——发生的过程作为过渡的结果

3. 软件规约的特性

- 正确性
- 完整性
- 无二义
- 一致性
- 可验证
- 非计算机专家可理解
- 可修改
- 可跟踪

CH11 设计概念

1. 设计工程创建软件的表示或模型

分析模型侧重于描述所需的数据、功能和行为
设计模型提供有关软件数据结构、体系结构、接口和组件的详细信息

2. 软件工程设计

- Data/Class design——将分析类转换为实现类和数据结构
- Architectural design——定义主要软件结构元素之间的关系
- Interface design——定义软件元素、硬件元素和最终用户如何通信
- Component-level design——转换结构元素到软件组件的过程描述中

3. 设计和质量

- 良好设计演化的特征
 1. 设计必须实现分析模型中包含的所有显式需求，并且必须满足客户所需的所有隐式需求
 2. 对于生成代码的人和测试并随后支持软件的人来说，设计必须是可读的、可理解的指南
 3. 设计应提供软件的完整图景，从实现的角度解决数据、功能和行为领域
- 质量指导原则

1. 架构

- 是使用可识别的架构风格或模式创建的
- 由表现出良好设计特征的组件组成
- 可以以进化的方式实现

2. 模块化——软件应在逻辑上划分为元素或子系统

3. 不同表示——含数据、架构、接口和组件

4. 导出数据结构——适用于要实现的类的数据结构，并从可识别的数据模式中提取

5. 导出显示独立功能特征的构件

6. 导出接口——降低组件之间以及与外部环境的连接复杂性

7. 可重复——使用由软件需求分析期间获得的信息驱动的可重复方法推导出设计

8. 可读性——使用有效传达其含义的符号来表示设计

○ 一些概念的含义

- 抽象——数据、过程（自下而上）
- 细化——详细阐述所有抽象的细节（自上而下）
- 架构——软件的整体结构
 - 架构的组成
 - 结构特性——定义系统的组件（例如，模块、对象、过滤器）以及这些组件的打包方式和相互交互的方式
 - 功能外的属性——说明设计架构如何达到对性能、容量、可靠性、安全性、适应性和其他系统特性的要求
 - 相关系统——在类似系统系列的设计中经常遇到的可重复模式
 - 架构的作用
 - 促进利益相关者的沟通
 - 有利于系统设计的早期决策
 - 易于理解的系统级抽象
 - 架构体现了系统的
 - 模块化
 - 抽象和信息隐藏
 - 接口设计
- 模式——“传达”经过验证的设计解决方案的本质
 - 模式的目的一一易于重用
 - 模式的种类
 - 架构模式——软件系统的基本结构组织形式或结构方案（整体）
 - 架构模式由一组预定义的子系统组成，这些子系统定义了它们的职责，并为组织和管理这些子系统提供了规则和指南（例如B/S， C/S）
 - 设计模式——为软件系统的子系统、组件或组件之间的关系提供了完善的解决方案（局部）
 - 设计模式描述了用于解决特定环境中常见软件设计问题的组件以及这些组件在相互通信时的可重现结构
 - 惯例——一种特定于编程语言的低级模式
 - 惯例描述了如何使用给定语言的特性来实现组件的特定方面或它们之间的关系
- 关注点分离——任何复杂的问题都可以更容易解决如果被分成小块
 - 关注点——软件需求模型的一部分指定的特性或行为
 - 通过将关注点分成更小，因此更易于管理的部分，解决问题需要更少的精力和时间

- 模块化——数据和功能的划分
 - 模块集成的成本使得模块并非越小越好
- 隐藏——受控的接口
 - 减少信息暴露
 - 减小修改影响
 - 强调接口通信
 - 减少全局变量
 - 鼓励封装
- 功能独立——单一的功能和低耦合（高内聚、低耦合）
 - 内聚——内聚度是模块相对功能强度的指示（由高到低）
 1. 功能内聚——模块内部共同完成同一功能
 2. 分层内聚——组成同一层次
 3. 顺序内聚——按顺序执行
 4. 通信内聚——使用相同的输入输出
 5. 过程内聚——受同一控制流支配
 6. 时间内聚——特定时间共同运行
 7. 实用内聚——没有联系，但在同一组
 - 耦合——耦合是模块之间相对相互依赖的指示（由低到高）
 1. 非直接耦合——相对独立工作
 2. 数据耦合——传递简单数据
 3. 标记耦合——传递数据结构
 4. 控制耦合——模块间有控制关系
 5. 外部耦合——共用简单数据
 6. 共用耦合——共用一部分数据环境
 7. 内容耦合——直接访问另一模块内部
 - 功能独立的重要性
 - 模块开发者专注于一个相对独立的部分
 - 不需要过多担心其他模块
 - 修改和错误的影响范围是本地化的
 - 单个模块更容易重用
 - 独立模块更容易维护和测试
- 方面——一种理解全局需求如何影响设计的机制
 - A横切B说明不考虑A就不能完成B
- 重构——一种重组技术简化设计
 - 重构是改变软件系统的过程，它不会改变代码的外部行为，但会改善其内部结构
 - 重构的原因
 - 冗余
 - 未使用的设计元素
 - 低效或不必要的算法
 - 构造不良或不适当的数据结构
 - 或任何其他可以纠正以产生更好设计的设计故障
- OO 设计概念
 - 设计类
 - 实体类
 - 控制类
 - 边界类
 - OO的三个基本特征：封装、继承、多态
- 设计类 — 提供能够实现分析类的设计细节

- 良好的设计类的特性
 - 完整性和充分性——完整地封装类中的属性和方法
 - 原始性——一个方法只关注于实现一个服务，不要重复
 - 高内聚——小型、专注、单一的类
 - 低耦合——类协作保持在最低限度
- 分析类——>实体类
- 界面——>边界类
- 对象生成、交互——>控制类
- 设计类的元素
 - 数据元素
 - 数据模型 --> 数据结构
 - 数据模型 --> 数据库架构
 - 架构元素
 - 应用领域
 - 分析类、它们的关系、协作和行为被转化为设计实现
 - 模式和“风格”
 - 接口元素
 - 用户界面 (UI)
 - 与其他系统、设备、网络或其他信息生产者或消费者的外部接口
 - 各种设计组件之间的内部接口
 - 构件元素
 - 部署元素

CH12~14 设计方式

CH12 行为建模

1. 什么是架构

程序或计算系统的软件架构是系统的一个或多个结构，它包括软件组件、这些组件的外部可见属性以及它们之间的关系

2. 架构有什么用

- 分析设计在满足其规定要求方面的有效性
- 在进行设计更改仍然相对而言的阶段考虑架构替代方案容易
- 降低与软件构建相关的风险

3. 架构的重要性

- 软件体系结构的表示是对基于计算机的系统的开发感兴趣的所有各方（利益相关者）之间进行交流的促成因素
- 架构突出了早期的设计决策，这些决策将对随后的所有软件工程工作产生深远的影响，对系统作为一个操作实体的最终成功产生深远影响
- 架构“构成了一种相对较小的、智力上可掌握的系统结构，便于理解

4. 架构描述——记录架构的产品集合（IEEE）

5. 架构风格

- 架构风格定义了
 - 一组执行系统所需功能的构件

- 一组连接器，可以实现“通信、协调和组件之间的合作”
- 定义组件如何集成以形成系统的约束
- 语义模型，使设计人员能够通过分析系统组成部分的已知特性来理解系统的整体属性
- 架构风格的种类
 - 数据流架构——批处理、管道和过滤器
 - 调用和返回架构——主程序/子程序、面向对象系统、分层系统
 - 独立组件架构——事件系统、触发器、监视器
 - 虚拟机架构——解释器，基于规则的系统
 - 仓库架构——数据库系统，黑板系统等

6. 架构模式

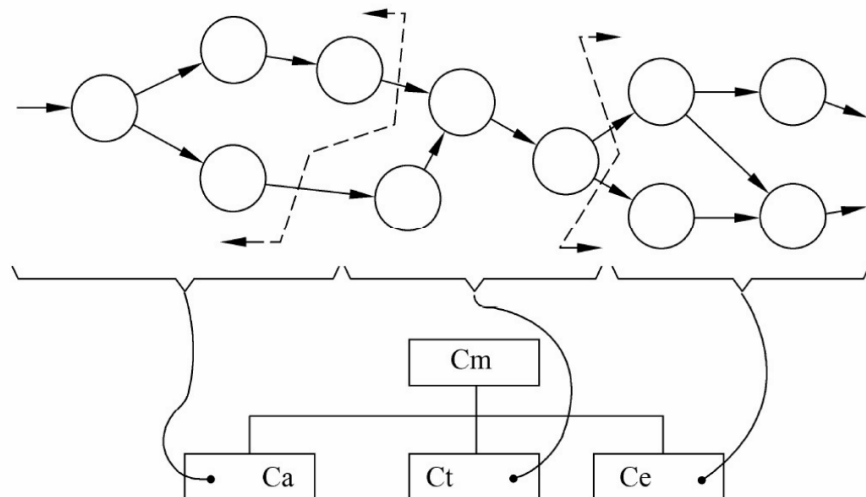
- 并发——并行执行任务（OS进程管理、任务调度）
- 持久性——数据在创立之后持久存在（数据库）
- 分发——系统在分布式环境中通信（代理）

7. 架构设计

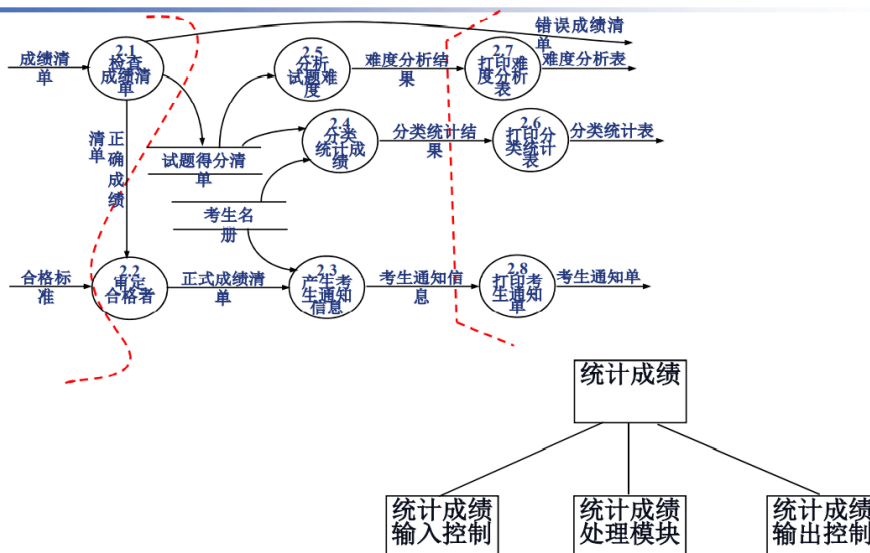
- 软件的上下文
 - 结构化设计

结构图

Structural Design

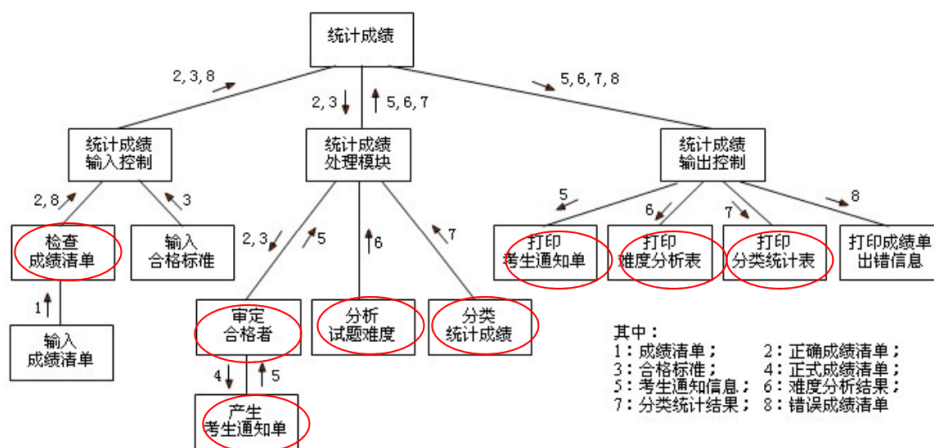


示例：统计成绩子图的第一级分解



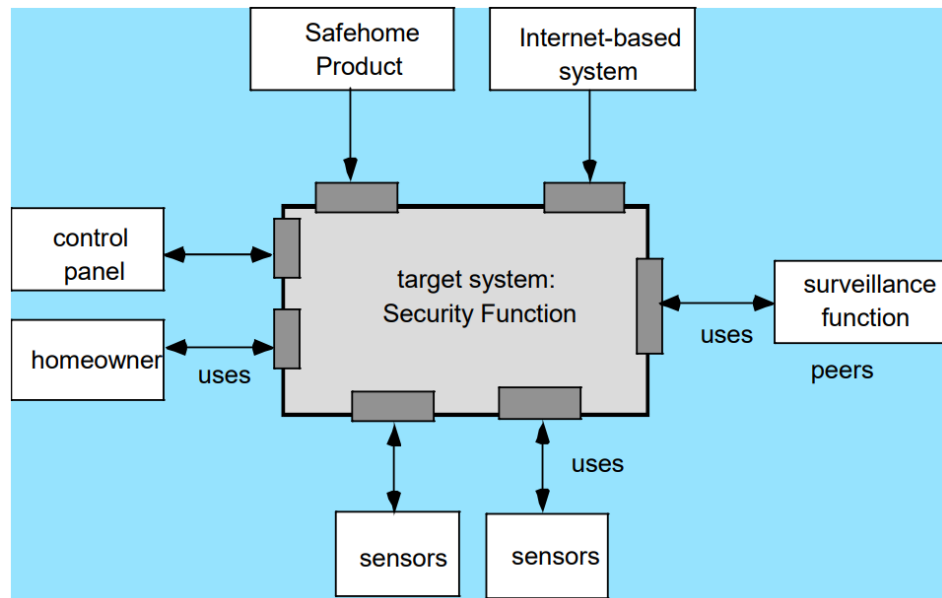
“统计成绩”第一级分解的结构图

“统计成绩”第二级分解的结构图

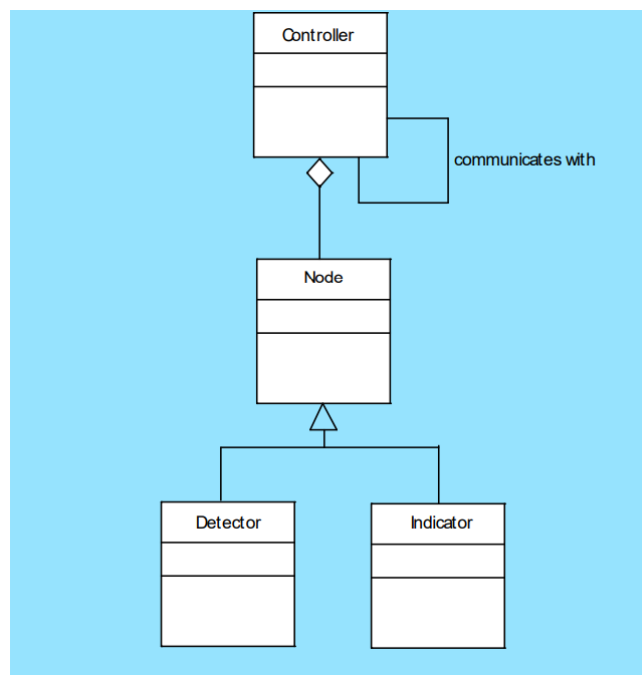


■ 架构上下文图 (ACD)

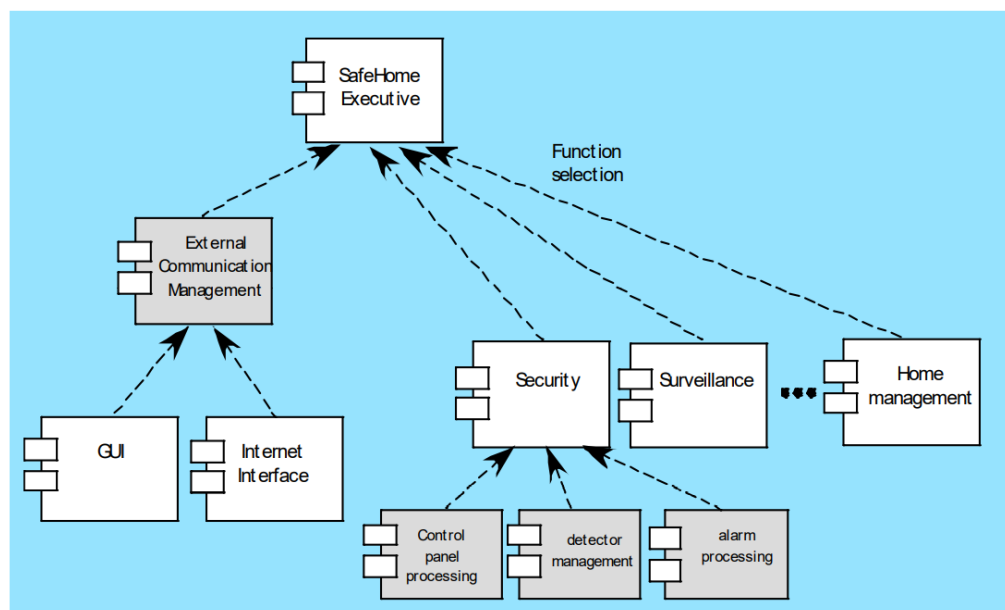
- 上级系统
- 使用者
- 下级系统



○ 架构原型



○ 构件精化



8. 架构的考虑因素

- 经济——最好的软件是整洁的，依靠抽象来减少不必要的细节
- 可见性——架构决策及其原因对于稍后检查模型的软件工程师来说应该是显而易见的
- 间距——在设计中分离关注点而不引入隐藏的依赖关系
- 对称——架构对称意味着系统在其属性上是一致的和平衡的
- 涌现——涌现的、自组织的行为和控制

9. 架构设计文档

1. 确定每个决策需要哪些信息项
2. 定义每个决策和适当要求之间的联系
3. 当需要评估替代决策时，提供改变状态的机制
4. 定义决策之间的先决关系以支持可追溯性经常使用基于经验的评审、原型评估、情景评审和清单
5. 将重大决策与决策产生的架构视图联系起来
6. 记录并传达所有决策

10. 架构描述语言（ADL）——分解架构组件、将单个组件组合成更大的架构块、表示接口（组件之间的连接机制）

11. 架构评估

1. 评估软件架构满足系统质量要求和识别潜在风险的能力
2. 通过及早发现设计问题，有可能降低项目成本
3. 经常使用基于经验的评审、原型评估、情景评审和清单

12. 敏捷开发与架构

- 为了避免返工，用户故事用于在编码之前创建和发展架构模型
- 软件架构师将用户故事放在不断发展的故事板
- 运行良好的敏捷项目包括在每个冲刺期间交付工作产品
- 审查从冲刺中出现的代码可能是一种有用的架构审查形式

CH13 构件级设计

1. 什么是构件

一个模块化的、可部署的、可替换的系统部分，它封装了实现并公开了一组接口。——OMG
从OO角度来看——一系列相互协作的类

2. 构件定义了

- 数据结构
- 算法
- 接口特征
- 通信机制

3. 构件的设计原则

1. **开闭原则（OCP）**——一个模块 [组件] 应该对扩展开放但对修改关闭
2. **里氏替换原则（LSP）**——“子类应该可以替代它们的基类。
3. **依赖倒置原则（DIP）**——依赖于抽象。不要依赖实体
4. **接口分离原则（ISP）**——许多专用的接口都比一个通用接口好
5. **发布重用等效原则（REP）**——重用的粒度就是发布的粒度
6. **共同闭合原则（CCP）**——同时修改的类应该放在一起
7. **共同重用原则（CRP）**——不能同时复用的类不应该放在一起

4. 进行构件级设计

1. 识别与问题域对应的所有设计类
2. 识别与基础设施领域相对应的所有设计类
3. 细化所有不作为可重用构件的设计类——明确协作信息、接口、属性、操作

4. 描述持久性数据源（数据库和文件）并确定管理它们所需的类
5. 开发并细化类或组件行为表示
6. 细化部署图并提供实现细节
7. 考虑每个组件级设计表示并始终考虑替代方案

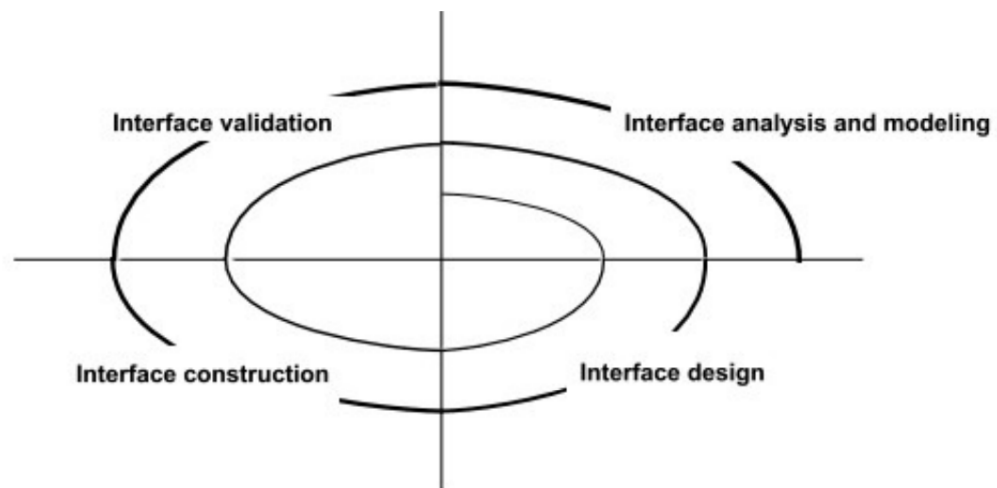
CH14 用户接口设计

1. 用户接口设计的黄金规则

- 用户控制操作
 - 以不强迫用户进行不必要或不希望的操作的方式来定义交互模式
 - 提供灵活的交互方式
 - 允许用户交互可中断和可撤消
 - 随着技能水平的提高，简化交互并允许定制交互
 - 对用户隐藏内部技术
 - 设计与屏幕上出现的对象直接交互
- 减少用户记忆负担
 - 减少对短期记忆的需求
 - 建立有意义的默认值
 - 定义直观的快捷方式
 - 界面的视觉布局应该基于真实世界
 - 以渐进的方式披露信息
- 保存界面一致
 - 允许用户将当前任务放入有意义的上下文中（明确当前的界面是做什么的）
 - 在一系列产品中保持一致性
 - 如果过去的交互式模型已经创造了用户期望，除非有令人信服的理由，否则不要进行更改

2. 用户界面设计过程

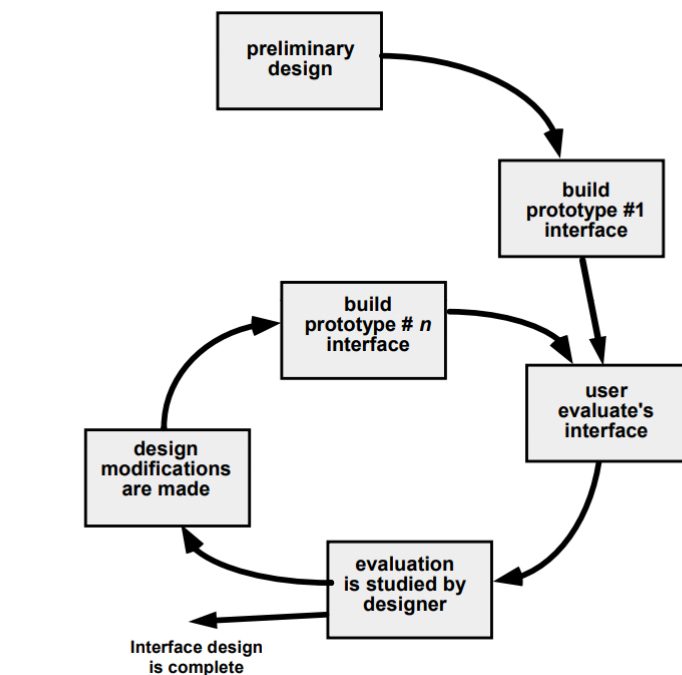
- 界面分析和建模
- 界面设计
- 界面构造
- 界面确认



3. 界面分析

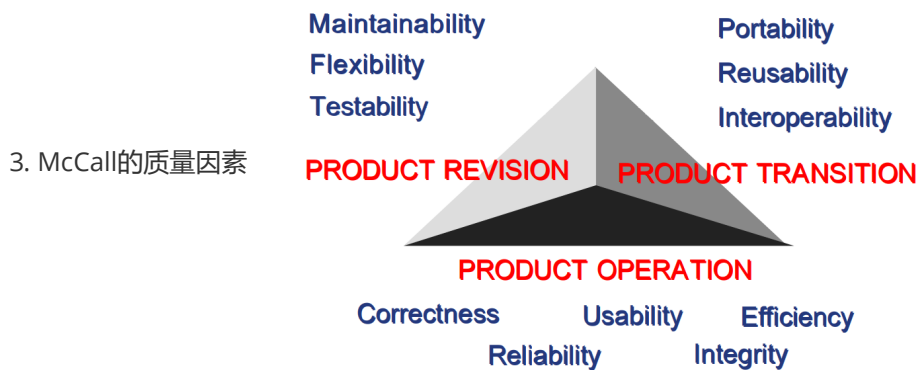
- 最终用户
- 执行的任务（可以使用泳道图）

- 界面的内容
 - 被使用的环境（办公室等）
4. 用户界面设计模型
- 用户模型——系统所有最终用户的概况
 - 设计模型——用户模型的设计实现
 - 心理模型——用户对界面的映像
 - 实现模型——具体的实现方式
5. 界面设计步骤
- 使用在界面分析过程中开发的信息，定义界面对象和动作（操作）
 - 定义将导致用户界面状态改变的事件（用户操作）。为这种行为建模
 - 描述每个界面状态
 - 说明用户如何根据通过界面提供的信息来解释系统的状态
6. 界面设计时的问题
- 响应时间
 - 帮助
 - 错误处理
 - 菜单和命令
 - 国际化
7. 设计评估



CH15~16 软件质量

1. 什么是软件质量——一个**有效的软件过程**，其应用方式创造了一个**有用的产品**，为生产它的人和使用它的人提供可衡量的价值。
2. 用户满意度 = 合规产品 + 质量好 + 在预算和计划内交付



1. 产品修改

1. 维护性
2. 灵活性
3. 易测试性

2. 产品转移

1. 可移植性
2. 可复用性
3. 互操作性

3. 产品运行

1. 正确性
2. 易用性
3. 效率
4. 可靠性
5. 完整性

4. ISO9126质量因素

- 功能性
- 可靠性
- 易用性
- 效率
- 维护性
- 可移植性

5. 软件质量的困境——市场机会和产品质量不能两全

6. “足够好”的软件——解决用户的关键问题、后续修复已知的bug

7. 质量成本（逐渐增加）

1. 预防成本（COP）
2. 评估成本（COA）
3. 内部失败成本
4. 外部失败成本

实现软件质量（SQA）

1. SQA的方法

1. 项目管理——项目计划包括质量和变更管理的明确技术
2. 质量控制——用于确保工作产品符合其规范的一系列检查、审查和测试
3. 质量保证——包括审计和报告程序，用于向管理层提供做出主动决策所需的数据

2. SQA的元素

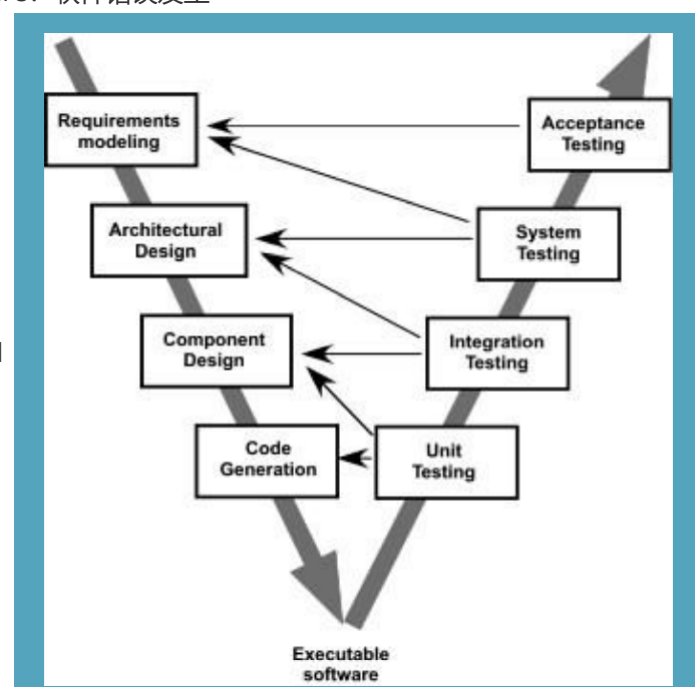
1. 标准
 2. 审查和审核
 3. 测试
 4. 错误/缺陷收集和分析
 5. 变更管理
 6. 教育
 7. 供应商管理
 8. 安全管理
 9. 安全性
 10. 风险管理
3. SQA小组的作用
1. 为项目准备 SQA 计划
 2. 审查**过程描述**是否符合组织政策、内部软件标准、外部强制标准（例如，ISO-9001）和软件项目计划的其他部分
 3. 审查**软件工程活动**是否符合定义的软件过程
 4. 审核指定的**软件工作产品**是否符合定义为软件过程的一部分的产品
 5. 确保软件工作和工作产品中的偏差被记录并根据文件化程序进行处理。
 6. 记录任何违规行为并向高级管理层报告
4. SQA的目标
1. 需求质量
 2. 设计质量
 3. 代码质量
 4. 质量控制的有效性

CH17~19 测试策略和技术

1. Error, Fault and Failure

- Error: 设计或编程中的错误；系统没有产生预期的输出
- Fault: 代码中的错误；Error的结果
- Failure: 软件错误发生

2. V—Model



3. Verification vs. Validation（角度、标准不同）

- Verification（局部）——确保软件正确实现特定功能的一组任务

- Validation（整体）——确保已构建的软件可追溯至客户要求

4. 测试策略——从小到大

- 单元测试
- 集成测试
- 系统测试
- 验收测试

5. 测试的步骤

1. 在开发测试之前，我们应该确定测试的目标（例如，可靠性、功能性）
2. 决定如何进行相关测试，决定哪种测试最合适，需要使用什么样的测试项目
3. 开发测试用例
4. 确定测试的预期结果
5. 执行测试用例
6. 将测试结果与预期结果进行比较

6. 测试使用的替代品

1. stub——代替底层

- 模拟目标硬件相关函数
- 模拟要实现的函数

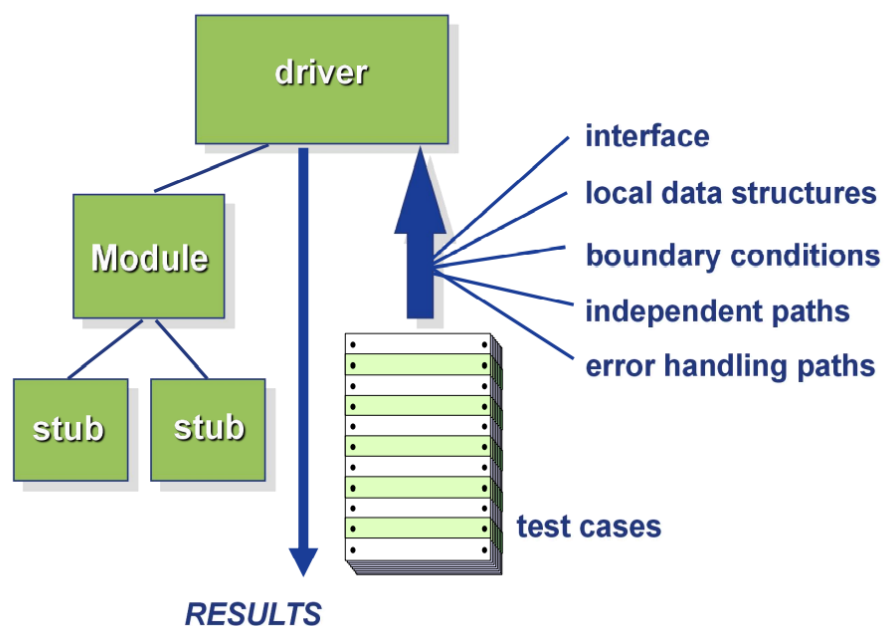
2. driver——代替顶层

- 提供了一个框架用于设置输入参数，环境，执行单元
- 读取输出参数（保存结果并检查它们是否正确）

单元测试

测试对象：

- 接口
- 局部数据结构
- 边界条件
- 独立路径
- 错误处理路径



集成测试

1. 集成测试的目的

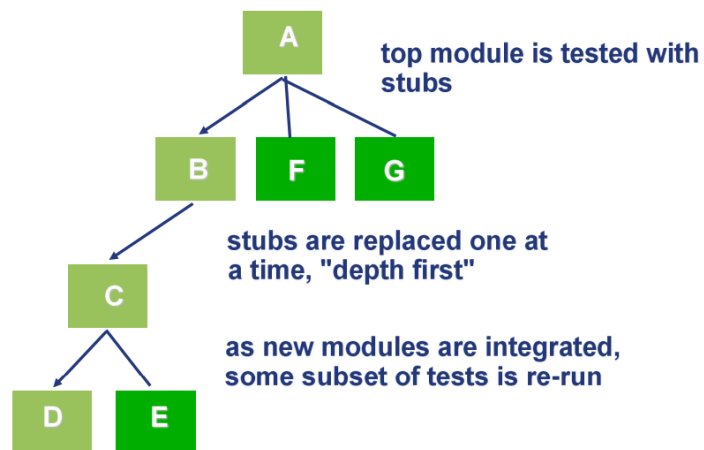
- 验证组件与系统的其余部分是否正常工作
- 验证组件之间的交互
- 检查组件/子系统接口之间交换的数据是否符合规范

2. 集成测试的策略

1. Big Bang

2. 增量集成

- **自顶向下** (深度或者广度优先)



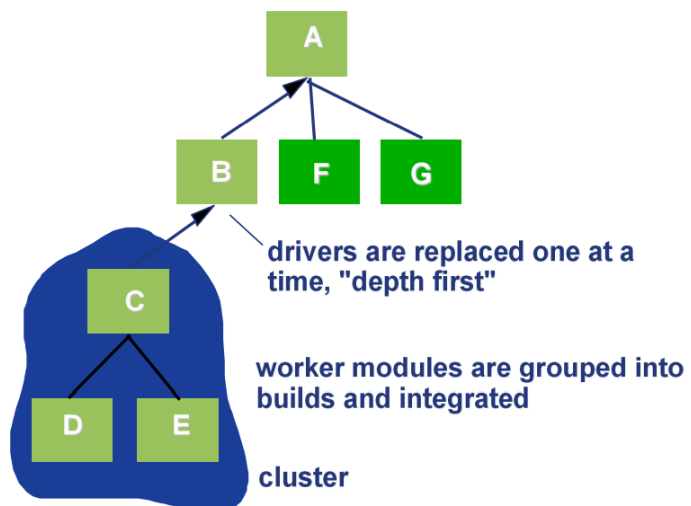
- **优势**

- 程序的骨架版本可以早期存在并允许演示
- 设计错误可能会更快被发现
- 减少对测试驱动程序的需求
- 故障定位更容易

- **缺点**

- stub的构建成本可能很高

- **自底向上**



- 优势
 - 自下而上测试的测试用例可以仅根据功能设计信息设计，不需要结构设计信息。当详细的设计文档缺乏结构细节时，自下而上的方法很有用
 - 特别适用于对象和重用。
- 缺点
 - 在添加最后一个模块之前，程序作为一个整体是不存在的
 - 自下而上的测试需要测试driver，而不是测试stub
 - 随着测试在层次结构上进行，自下而上的测试变得更加复杂，因此开发和维护成本更高；实现良好的结构覆盖变得更加困难
- 三明治测试——工作模块被集成成cluster变成大模块
- 回归测试——加入新模块后重新进行原来部分的测试
- 冒烟测试——产品每天构建

面向对象测试

- 类测试就是单元测试
- 集成测试分成
 - 基于线程的测试——集成响应一个输入或响应所需的一组类事件
 - 基于使用的测试——集成响应一个用例所需的一组类
 - 集群测试——集成演示一次协作所需的一组类

系统测试——充分运用基于计算机的系统

系统测试的目的

- 在将要运行的真实环境中测试整个系统
- 确保系统满足整体工作的要求
- 还要测试系统功能之外的方面
- 结果有时用于系统验收
- 验证软件用户手册
- 估计可靠性和可维护性

验收测试——关注软件需求

产出

- 功能或性能符合规范
- 使用缺陷列表来描述与规范的偏差

Alpha/Beta测试——关注客户使用情况

Alpha 测试由最终用户在开发人员站点进行

Beta 测试在最终用户站点进行

软件测试方法

1. 黑盒测试（功能化）
 - 用例基于软件规约
 - 可能会发现与白盒测试不同的错误类别
 - 倾向于在测试的后期阶段应用

- 查找的错误
 - 不正确或缺少函数
 - 接口错误
 - 数据结构或外部数据库访问错误
 - 行为或性能错误
 - 初始化和终止错误

2. 白盒测试（结构化）

- 用例基于代码
- 为什么要结构化测试

避免没测试到的部分产生问题
覆盖低层次的细节

- 执行方式
 - 保证模块内的所有独立路径至少被执行一次
 - 执行所有逻辑决策的true和false
 - 在其边界和操作范围内执行所有循环界限
 - 检验内部数据结构以确保其有效性

3. 测试方法

Black Box Methods	White Box Methods
Equivalence Partitioning （等价类划分） Boundary Value Analysis （边界值分析） Graph-Based Methods （因果图分析） Orthogonal Array Testing （正交数组法）	Desk Checking （桌面检查） Walkthrough （代码走查） Inspection （代码审查） Basis Path Testing （基本路径测试）

Static Testing

4. 静态测试和动态测试

- 静态测试——检查
- 动态测试——运行

CH20~21安全工程和SCM（软件配置管理）

1. 为什么需要安全工程

- 保护系统资产免受攻击
- 修bug难以提升安全性

第一定律： 无论你处于系统生命周期的哪个阶段，系统都会发生变化，而改变它的需求会贯穿整个生命周期

2. 软件配置管理和软件支持的差别

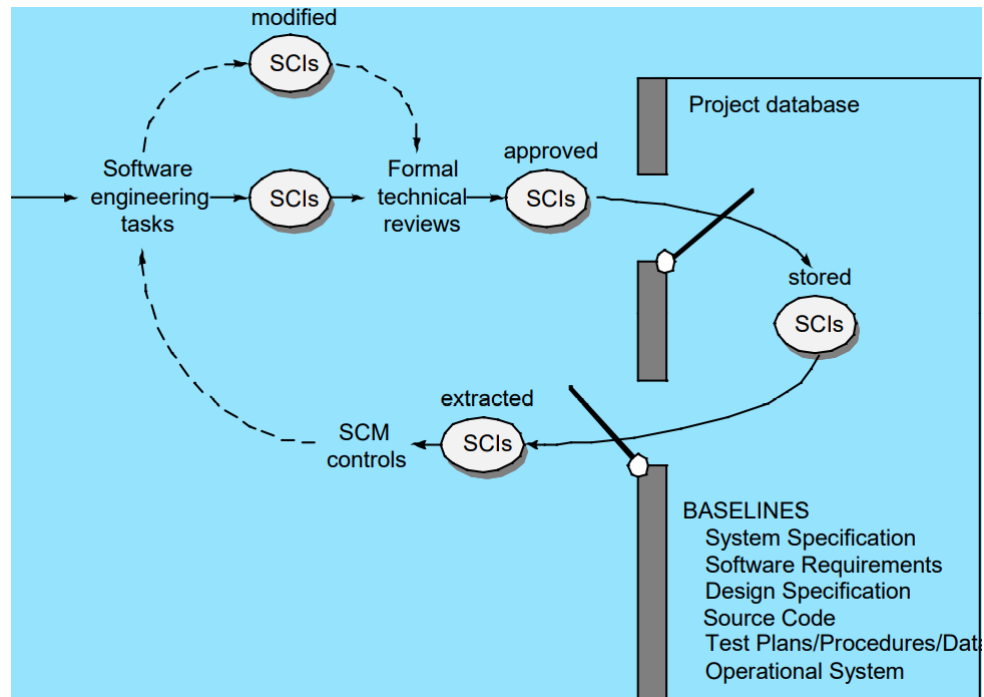
- 软件支持是发生在交付给客户并投入运行后的活动
- 软件配置管理是软件项目开始时就启动，到软件淘汰时才终止的活动

3. 软件配置包含三个方面

1. 程序
2. 文档
3. 数据

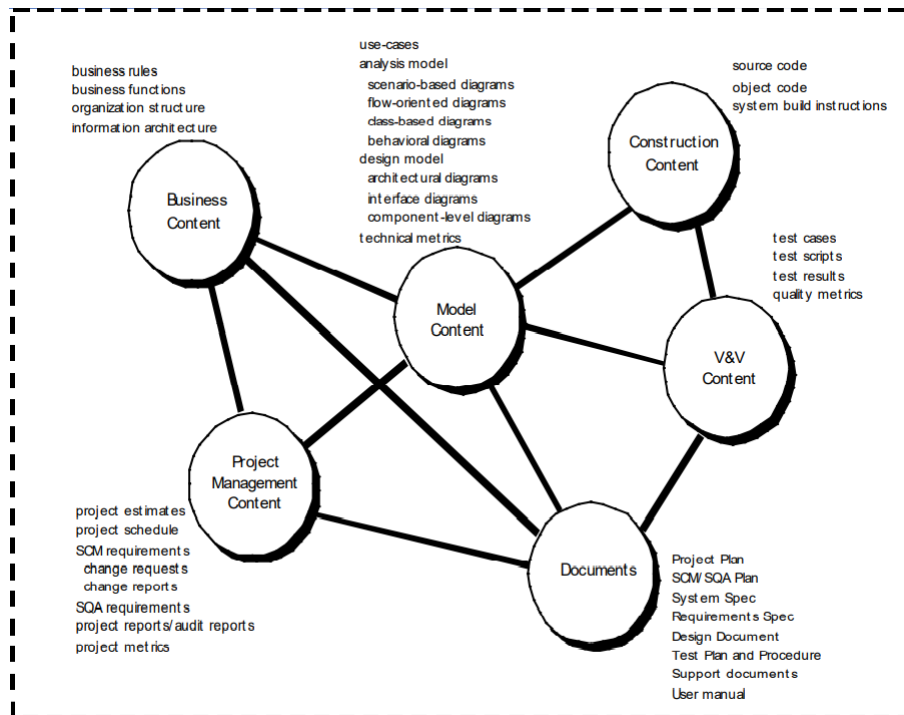
4. 基线——已正式审查并达成一致的规范或产品，可作为进一步开发的基础，并且只能通过正式的流程才能更改

- 基线是软件开发中的一个里程碑，其标志是交付一个或多个通过正式的技术审查获得批准的软件配置项（SCI）



5. SCM数据库

- 存放各种数据



6. SCM特征

- 版本控制——保存所有这些版本，以实现对产品发布的有效管理，并允许开发人员返回到以前的版本
- 依赖跟踪——存储库管理存储在其中的数据元素之间的各种关系

- 需求跟踪——跟踪由特定需求规范产生的所有设计和施工组件和可交付成果
- 配置管理——特定项目里程碑或生产版本的一系列配置
- 审核跟踪——建立关于何时、为什么以及由谁进行更改的信息

7. SCM元素

- 构件元素——一组具有数据库功能的工具，使人们可以访问和管理每个软件配置项
- 过程元素——一个动作和任务集，定义变更管理的方法
- 构造元素——一组自动软件构造工具，确保正确装配所有构件
- 人员元素——为了实施有效的 SCM，软件团队使用一组工具和流程特征（包括其他 CM 元素）

8. SCM过程（CCA:变更控制授权人 ECO：工程变更工单）

- 5个SCM任务
 - 标识
 - 版本控制——结合程序和工具来管理在软件过程中创建的不同版本的配置对象
 - 版本控制系统实现或直接集成了四个主要功能
 1. 存储所有相关配置对象的项目数据库（存储库）
 2. 存储配置对象的所有版本
 3. 使软件工程师能够收集所有相关配置对象并构建软件的特定版本的制作工具
 4. 问题跟踪（也称为错误跟踪）功能，使团队能够记录和跟踪与每个配置对象相关的所有未决问题的状态
 - 变更控制

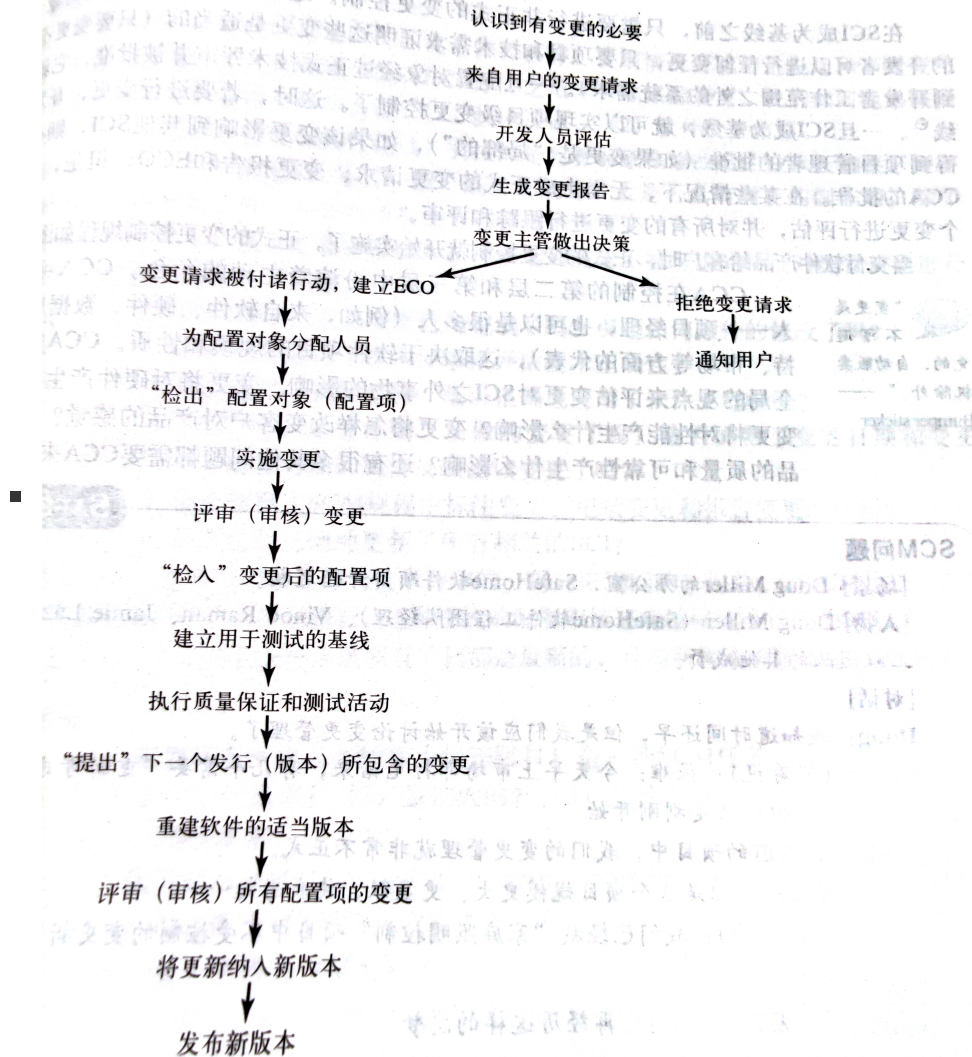
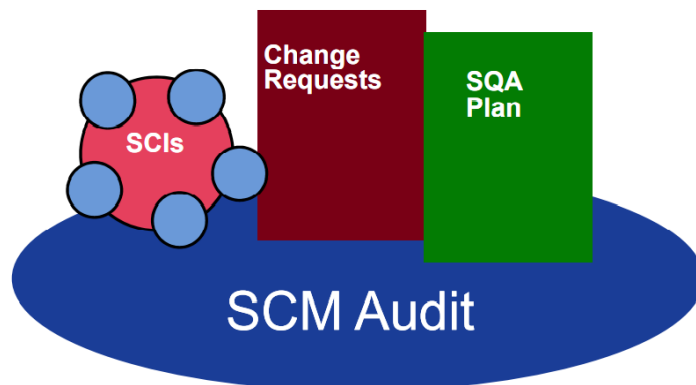


图17-5 变更控制过程

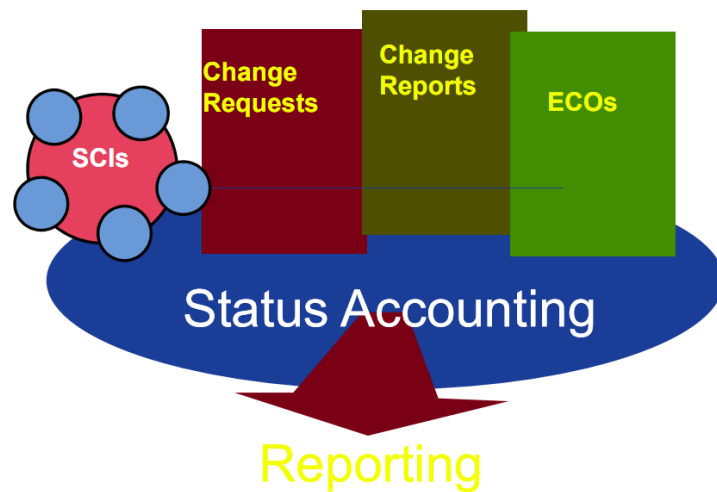
■ 配置审核

Auditing

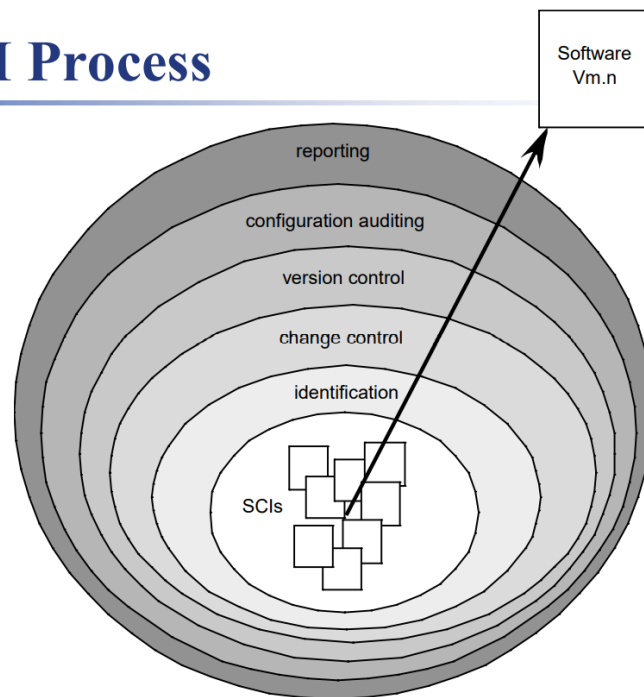


■ 报告

Status Accounting



The SCM Process



CH22 项目管理概念

1. 项目管理的范围——4P: People、Product、Process、Project

1. 软件利用相关者:

- 高级经理——决定对项目有重大影响的业务事件
- 项目（技术）经理——计划、激励、组织和控制员工
- 参与者——提供设计产品或应用程序所必需的技术技能
- 客户——指定要设计的软件的要求
- 最终用户——与软件交互的人

2. 产品范围

- 上下文——要构建的软件如何适应更大的系统、产品或业务环境，以及由于环境而施加了哪些约束
- 信息目标——用户可以看见的数据输入、数据输出

- 功能和性能——软件执行什么功能将输入数据转换为输出

软件项目范围必须在管理和技术层面明确且易于理解。

3. 问题分解——将复杂的问题分成若干更易处理的小问题

4. 过程

一旦建立了过程框架，就要

- 考虑项目特征
- 确定所需的严格程度
- 为每个软件工程定义一个任务集

任务集 = 软件工程任务 + 工作产品 + 质量保证点 + 里程碑

5. 项目

1. 项目处于危险的十个信号

1. 软件人员不了解客户的需求时
2. 产品范围定义不明确
3. 变更管理不善
4. 所选技术发生变化
5. 业务需求变化[或定义不明确]
6. 截止日期是不现实的
7. 用户很抗拒
8. 赞助丢失[或从未真正获得]
9. 项目团队缺乏具备适当技能的人员
10. 管理者[和从业者]没有利用最佳实践和经验教训

2. 常见的项目管理方法

1. 从正确的基础上开始工作——首先通过努力理解要解决的问题，然后设定现实的目标和期望来实现的
2. 保持动力——项目经理必须提供激励措施，将人员流动率控制在最低限度，团队应在其执行的每项任务中强调质量，高级管理人员应尽一切可能远离团队
3. 跟踪进度——作为质量保证活动的一部分，随着工作产品（例如模型、源代码、测试用例集）的生成和批准（使用正式的技术评审），进度会被跟踪。
4. 做出明智的决定——项目经理和软件团队的决定应该是“保持简单”
5. 进行事后分析——建立一个一致的机制来提取每个项目的经验教训

3. W5HH原则

1. Why is the system being developed?——为什么
2. What will be done? ——做什么
3. When will it be accomplished?——什么时候做
4. Who is responsible?——谁负责
5. Where are they organizationally located?——其他人的组织机构位于何处
6. How will the job be done technically and managerially?——如何完成技术工作和管理工作
7. How much of each resource (e.g., people, software, tools, database) will be needed?——每种资源需要多少

4. 关键实践

1. 正式的风险管理
2. 经验成本和进度估算
3. 基于度量的项目管理

4. 获得价值跟踪
5. 针对质量的缺陷跟踪
6. 人员计划管理

CH23 过程和项目度量

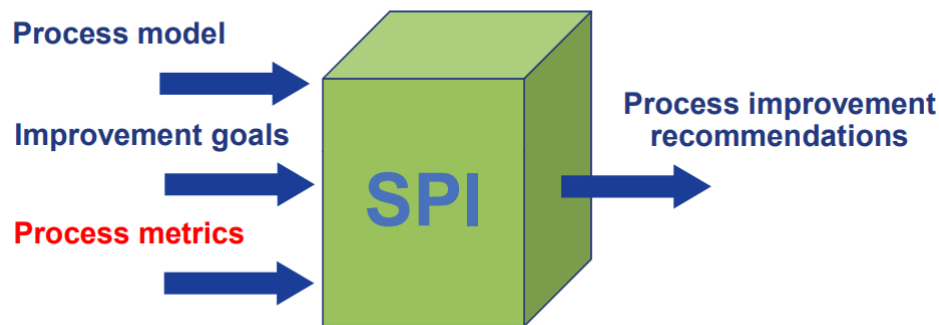
1. 度量的意义

1. 评估正在进行的项目的状态
2. 跟踪潜在风险
3. 在问题造成不良影响前发现风险
4. 调整工作流程或任务
5. 评估项目团队控制软件工作产品质量的能力

2. 过程度量——间接测量软件过程的效率

根据过程中获得的一系列数据或软件工程任务的特性来进行测量。

3. 软件过程改进 (SPI)



4. 过程度量的五个方面

- 质量相关——关注工作产品和可交付成果的质量
- 生产力相关——工作产品的生产与付出的努力相关
- 统计 SQA 数据——错误分类和分析
- 缺陷去除效率——传播从流程活动到活动的错误
- 重用数据——生产的组件数量及其可重用程度

5. 项目度量的三个方面

- 如期
- 质量
- 成本

6. 面向大小的指标——每千行代码的错误数、缺陷、成本等

7. 面向功能的指标——每个功能点的错误数、缺陷、成本等

功能点的好处：编程语言独立、数目容易确定、不会对不同实现方法产生影响、易于测量重用组件的影响

8. 面向对象的度量

- 场景脚本（用例）的数量
- 支持类的数量（实现系统所需但与问题域没有直接关系）
- 每个关键类的平均支持类数量（分析类）
- 子系统的数量（实现用户可见的功能的类的集合）

9. 软件度量规则

- 在解释指标数据时使用常识和组织敏感性
- 定期向收集度量和指标的个人和团队提供反馈
- 不要使用指标来评估个人
- 与开发者和团队合作，制定明确的目标和指标，用于实现这些目标
- 不要使用指标来威胁个人或团队
- 指出问题区域的度量数据不应被视为“负面”。这些数据只是流程改进的一个指标
- 不要执着于单一指标而排斥其他重要指标

10. 测试质量

- 正确性
- 可维护性
- 完整性
- 可用性

CH24~25 项目估算和调度

项目估算

项目任务设置

- 确定项目范围
- 确定任务的弹性
- 分析风险
- 估计所需资源
- 估算工作量
 - 最终——制定项目计划

1. 项目范围

- 项目范围的内容
 - 交付给最终用户的功能和特性
 - 输入和输出的数据
 - 由于使用软件而呈现给用户的“内容”
 - 性能、约束、接口、和约束系统的可靠性
- 定义范围的方法
 - 与所有利益相关者沟通后制定软件范围的叙述性描述
 - 由最终用户开发的一组用例

2. 工作分解结构 (WBS)——为计划和管理项目的进度、成本和变更，风险分析，组织结构，度量提供了基础

1. WBS的种类

- 产品WBS——面向实体
- 过程WBS——面向活动

2. 进行WBS的五种方法

1. 使用指南：一些组织提供的指南
2. 类比方法：审查类似项目的 WBS，并根据我们的项目 进行定制。
3. 自上而下的方法：从项目中最大的项目开始，并将它们分解
4. 自下而上的方法：从详细的任务开始，并将它们汇总
5. 思维导图方法：以非线性格式写下任务，然后创建 WBS 结构

3. 乐观—期望—悲观估计法

$$S = (S_{opt} + 4S_m + S_{pess})/6$$

基于代码行的估计

优点：易于测量、容易自动化

缺点：仅限于代码不能用于设计、依赖于语言、没有考虑功能的复杂性、与设计的好坏挂钩

基于功能点的估计

优点：可用于最早的需求阶段 § 独立于编程语言、产品设计或开发风格 § 用户视图，而不是实现视图 § 可用于衡量非编码活动 § 存在大量历史数据 § A有据可查的方法

缺点：无法直接计算现有产品（源代码）的 FP 内容 § 难以自动化 § FP 不反映语言、设计或风格差异 § FP 设计用于估计商业数据处理应用 § 主观计数

4. 经验估计模型

$$\text{工作量} = A + B * (e_V)^C \quad (\text{其中 } A、B、C \text{ 是经验常数，} e_V \text{ 是 } LOC \text{ 或 } FP)$$

项目调度

1. 项目为什么会延期

- 不切实际的截止日期
- 需求的变化
- 低估工作量
- 不可预测的风险
- 技术难题
- 人为困难
- 沟通不畅
- 未能认识到项目落后于计划或缺乏纠正问题的行动

2. 调度的原则

- 划分——定义多个不同的任务
- 相互依赖性——明确任务的相互关系
- 工作量确认——确保人力资源可用
- 确定责任——明确责任承担者
- 明确输出结果——确定活动产生的结果
- 确定里程碑——进行质量审查

3. 调度的步骤

1. 定义任务集——WBS
2. 安排活动
3. 绘制项目网络图
4. 关键路径分析
5. 使用甘特图进行调度
6. 进度跟踪

CH26 风险分析

1. 软件风险的种类

- 项目风险
- 技术风险

- 商业风险
- 2. 被动风险策略——项目团队在风险发生时对风险做出反应
- 3. 主动风险策略——纠正风险的根本原因4
- 4. 风险识别（有哪些风险）
 - 1. 产品规模
 - 2. 商业影响
 - 3. 客户特性
 - 4. 过程定义
 - 5. 开发环境
 - 6. 开发技术
 - 7. 人员才干及经验
- 5. 评估项目风险
- 6. 风险因素
 - 1. 性能风险
 - 2. 成本风险
 - 3. 支持风险
 - 4. 进度风险
- 7. 风险预测的方法
 - 1. 建立一个反映风险感知可能性的尺度
 - 2. 描述风险的后果
 - 3. 估计风险对项目和产品的影响
 - 4. 标明风险预测的总体准确性，以便不会有误会
- 8. RMMM:风险缓解监控与管理
 - 缓解——我们如何避免风险
 - 监控——我们可以跟踪哪些因素来确定风险是增加还是减少？
 - 管理——如果风险成为现实，我们有哪些应急计划？